

WOD2Sim: Separating Integration Failures from Policy Failures in Closed-Loop Driving Simulation

Alba Maria Tellez Fernandez

Abstract—Closed-loop driving evaluation can misattribute degraded rollout metrics to policy quality when the simulator boundary drops route geometry, changes timing assumptions, or produces incomplete evidence. WOD2Sim audits semantic, temporal, lifecycle, deployment, and evidence contracts between Waymo Open Motion Dataset-style trajectory policies and AlpaSim external drivers. We evaluate a retained 197-call AlpaSim trace using 15 single-fault mutations and 15 valid prefix controls; expected labels are withheld from the detector. WOD2Sim classifies 30/30 cases correctly, detects and localizes 15/15 faults, and produces 0/15 control false positives. An executable completion-and-metrics gate classifies 15/30 and detects no faults (exact paired McNemar $p = 0.000061$). Across 3,000 fault decisions, median post-trace diagnosis is 234.855 μ s. In 200 paired route-following measurements, online camera/context and freshness checks add 14.552 μ s median while preserving trajectory output, equivalent to 0.812% of the retained driver-call median. Separately, the contract gate rejects all 15 completed, metric-bearing command-only rollouts admitted by the status gate. The result is a measured failure-attribution method; learned-policy quality and scenario coverage are outside this study.

I. INTRODUCTION

Dataset-trained trajectory policies and closed-loop simulators expose different failure surfaces. A policy designed around the Waymo Open Motion Dataset (WOMD) interface [1] may assume logged observations, local route geometry, and a fixed trajectory horizon. A simulator service must also accept incremental messages, meet the runtime clock, remain alive across session events, and retain enough evidence to explain a result. AlpaSim exposes this difference through its external-driver boundary [2].

A wrapper can therefore make an integration defect look like poor driving. Replacing a local route polyline with a left/straight/right command removes decision-relevant input, yet the rollout may still terminate with collision and progress metrics. Timing-grid mismatches, stale observations, optional import failures, missing assets, and incomplete manifests create similar confounds. A completed process is not sufficient evidence that a score characterizes policy behavior.

WOD2Sim treats this as a system-integration and attribution problem. Its contribution is threefold:

- five explicit contracts define when a simulator rollout may be interpreted as policy behavior;
- an adapter, audit, and artifact pipeline localizes violations and preserves completed, blocked, and invalid rows;
- a controlled trace-mutation study measures localization, post-trace diagnosis time, and online guard overhead

Alba Maria Tellez Fernandez is an Independent Researcher. Email: amtellezfernandez@gmail.com.

TABLE I
CONTRACTS ENFORCED AT THE POLICY/SIMULATOR BOUNDARY.

Mismatch	Contract	Mechanism	Validation
Command-only route	Semantic	Preserve route geometry	route-source audit
Policy horizon/runtime grid	Temporal	Deterministic resampling	cadence tests
Script flow/session service	Lifecycle	Idempotent late-event handling	lifecycle tests
Implicit host state	Deployment	Materialized manifests	readiness checks
Process exit/evidence	Evidence	Audit-valid summaries	claim gate

against an executable status-only gate.

The design targets WOD-style trajectory interfaces, but the present evaluation uses only dependency-light constant-velocity and route-following policies. It does not evaluate a learned policy.

A. Failure Attribution Rule

A rollout permits policy-behavior attribution only if its semantic, temporal, lifecycle, deployment, and evidence contracts pass. A policy-failure attribution additionally requires a retained failure layer identifying the policy. Otherwise the row is an integration, precondition, or evidence failure. Thus a missing actor proxy is not a weak planner, a command-proxy route is not a poor route-conditioned policy, and an incomplete audit is not a collision result. This rule permits policy behavior to be examined after validation; it does not declare every degraded metric a policy failure.

II. CONTRACT-GATED INTEGRATION

Table I maps the five recurring boundary mismatches to enforcement mechanisms. The *semantic contract* preserves route geometry, ego state, camera availability, and actor fields at prediction time. A high-level route command may be diagnostic context, but it cannot substitute for a local polyline. The *temporal contract* validates horizons and timestamps, anchors interpolation at the current ego pose, deterministically resamples a finite $N \times 2$ trajectory, and recomputes headings on the runtime grid.

The *lifecycle contract* makes start, active, close, cleanup, duplicate close, and late events explicit. Recoverable warnings are distinguished from fatal service errors. The *deployment contract* records launch commands, scene and policy identifiers, timeout, endpoint, repository state, image identity, and gated inputs. The *evidence contract* requires unique run identifiers, immutable manifests, terminal status, route and sensor audits, failed-run retention, and hashes linking generated paper assets to aggregate rows.

Figure 1 shows the boundary. WOD2Sim reconstructs policy-facing state, preserves route geometry, resamples outputs, and isolates optional model discovery. On the runtime

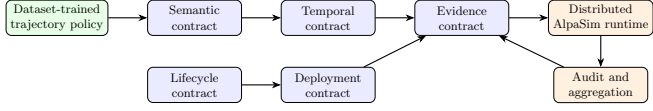


Fig. 1. Contract gates localize semantic, temporal, lifecycle, deployment, and evidence failures around the external-driver boundary.

side it records deployment and evidence state before a row can be attributed to policy behavior. Constant-velocity and route-following models form the public core. Learned-checkpoint and direct actor-aware entry points remain gated when their checkpoints or scene-matched actor proxies are unavailable.

III. IMPLEMENTATION AND EVIDENCE PIPELINE

A. Policy-Facing Adapter

The external driver maintains isolated state for each simulator session. Image, egomotion, route, and close messages update that state; prediction reads one consistent snapshot rather than reconstructing inputs from process-global variables. The route converter carries simulator waypoints into the policy input and records their source. When the explicit command-only diagnostic is selected, it labels the fallback as `command.proxy`; the audit can then reject the row even if the service and simulator finish normally. Static hazards remain geometry-bearing objects, while dynamic hazards retain position, motion, heading, and shape fields when the external interface supplies them. Visibility-risk signals remain diagnostics and do not create synthetic obstacles.

The temporal adapter maps a source trajectory to runtime times $t_k = k/f_{\text{run}}$ over the requested horizon. The current ego origin is prepended before interpolation, source timestamps must be finite and strictly increasing, and the output must be a finite $N \times 2$ array. Matching grids preserve samples; mismatched grids use deterministic interpolation, after which headings are recomputed from successive runtime points. Invalid horizons, frequencies, grids, or trajectories fail the temporal contract rather than reaching vehicle control. These behaviors are covered by dependency-light tests, but the scene-level temporal comparison remains blocked and is not reported as a result.

B. Artifact Lineage

Figure 2 shows the evidence path. Before launch, the matrix runner expands a configuration row and materializes its command, environment, scene identity, policy identity, adapter mode, prerequisite state, and expected route source. A readiness check distinguishes an executable row from a missing checkpoint, actor proxy, asset, or runtime dependency. Terminal state is then recorded as completed, failed, blocked, or planned; non-completed rows are retained rather than removed from the denominator.

For completed simulator rows, the audit reconstructs route-source and sensor-freshness status from retained telemetry and loads behavior metrics without using them to decide route validity. Aggregation rejects duplicate completed run identifiers, preserves failure codes, forms paired rows by

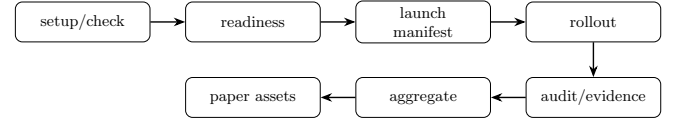


Fig. 2. Manifests and audits precede aggregation, so blocked or invalid rows remain visible and cannot silently become paper results.

policy, scene, and execution identifier, and writes CSV, JSON, LaTeX tables, and vector figures. Generated paper assets carry the aggregate data hash, and the submission validator compares their values with the JSON source. This lineage prevents manual table edits from changing a paper result independently of the row evidence. Restricted sensor frames and licensed scenes are not redistributed; the public package retains manifests, aggregate metrics, audit fields, and a stable frame schema. Consequently, artifact lineage is public, but complete replay still depends on legitimately acquired external scenes and runtime software.

IV. EVALUATION METHOD

The contract-validation matrix (CVM) contains 148 configured rows: 45 core, 30 semantic-ablation, 18 temporal-ablation, 40 lifecycle, and 15 fault-injection rows. It completes 115 rows, including 60 simulator rollouts; 33 optional-extension rows remain blocked.

The public core executes two dependency-light policies over 15 locally cached front-camera scenes. The semantic matrix pairs full-route and command-only variants of route following on the same scenes. The launcher does not expose a simulator seed override, so each policy/scene arm has one execution; the configured seed is provenance metadata rather than proof of deterministic replay. No authoritative metadata verifies straight, intersection, lane-change, dense-traffic, occlusion, or merge labels: 15 scenes remain unclassified and 0/6 required categories are verified.

The diagnostic experiment starts from a hashed, retained external AlpaSim trace with 197 drive calls. Fifteen single-field or runtime-context mutations cover three semantic, three temporal, three lifecycle, one plugin, three deployment, and two evidence faults. Fifteen increasing trace prefixes, each terminated by the retained close event, serve as valid controls. Mutation construction and diagnosis are separate: the detector receives only mutated events and runtime context; the scorer retains the expected label. For a paired comparator, define the executable status-only gate

$$B_{\text{status}}(r) = 1[\text{completed}(r) \wedge \text{metrics}(r)]. \quad (1)$$

Both methods return a decision for all 30 cases. We report correct classification, fault recall, localization, control false positives, Wilson 95% intervals, and an exact two-sided McNemar test over paired correctness.

Post-trace timing uses 200 randomized iterations, five calls per batch, yielding 3,000 fault-diagnosis and 6,000 all-case WOD2Sim measurements. A second 200-iteration paired ablation compares the same route-following prediction with camera shape/context and sensor-freshness guards enabled

TABLE II
DEPENDENCY-LIGHT PUBLIC-CORE INTEGRATION STATUS.

Public core policy	Rows	Done/att.	Audit	Route	Sensor	Crash	Blocked
constant_velocity	15	15/15	14/15	14/15	15/15	0	0
route_following	15	15/15	14/15	14/15	15/15	0	0

TABLE III
PAIRED CONTROLLED-TRACE DIAGNOSTICS. FP DENOTES A VALID CONTROL CLASSIFIED AS FAULTY.

Gate	Correct	Detected	Localized	FP
WOD2Sim	30/30	15/15	15/15	0/15
Status-only	15/30	0/15	0/15	0/15

or disabled, randomized within each iteration and batched over 20 calls. Both paths retain route extraction, trajectory generation, resampling, and reasoning serialization; output trajectories must match exactly. Completed simulator rows are separately audited for route source and sensor freshness. The same status gate is applied to metric-bearing command-only rows, while WOD2Sim additionally requires all five contracts.

V. RESULTS

Table II reports integration status rather than behavior scores. All 30/30 public-core rows complete with no service crash or blocker; 28 are audit-valid. The two invalid core rows and one invalid semantic full-route row come from the same scene, where the audit finds late command-proxy fallback. They remain integration/evidence failures despite successful process completion.

Controlled-trace results appear in Table III. WOD2Sim is correct on 30 of 30 cases, detects and localizes all 15 faults, and flags none of the 15 controls. The status gate is correct on 15 cases because it accepts every control and every completed fault case; it detects no faults. All 15 discordant pairs favor WOD2Sim (exact two-sided McNemar $p = 0.000061$). The Wilson 95% lower bounds are 0.886 for case classification and 0.796 for fault recall, making the finite denominators explicit.

Median WOD2Sim decision time is 207.980 μ s over 6,000 measurements; correct fault diagnosis is 234.855 μ s over 3,000 fault measurements. The paired online guard increment is 14.552 μ s, or 26.648% relative to the deliberately small unchecked route-following path. For operational context, it is 0.812% of the retained external driver-call median (1.793 ms); all 197/197 retained calls meet the 100-ms target.

The closed-loop invalidation check in Table IV is consistent with the mutation study. All 15/15 command-only rows satisfy B_{status} , whereas WOD2Sim rejects 15/15 as route-invalid. Thus metric-bearing completion alone would admit records whose route contract is known to be false.

Only 14/15 route-loss pairs satisfy the comparison criterion. Their full-minus-command means are mixed (progress -0.052, collision-any 0.071, plan deviation 0.016); they describe retained outcomes rather than a policy effect because each arm has one execution and no effective seed control.

TABLE IV
CLOSED-LOOP ATTRIBUTION CHECKS FROM RETAINED ROLLOUT AND AUDIT ARTIFACTS.

Closed-loop check	Obs.	Denom.	Meaning
Full-contract audit-valid	42	45	audit passed
Comparison-eligible pairs	14	15	valid/invalid arms
Status-only baseline accepted	15	15	completion + metrics
Invalid route rows rejected	15	15	route gate
Verified scenario categories	0	6	coverage withheld

Across all retained rows, 42 are policy-behavior-attributable diagnostics, 0 are policy-failure-attributable, and 33 are integration/precondition blockers. The remaining 106 include blocked, contract-invalid, synthetic, or benchmark-incomplete evidence. These counts operationalize the distinction: degraded metrics cannot assign policy failure unless the contracts pass and the retained failure layer is policy.

A. Interpretation

The controlled experiment validates classification and localization for the declared mutation set. Its 15 clean prefixes supply a negative class, so the observed 0/15 false-positive count is independently scored rather than defined by audit admission. The closed-loop rows test a second path: adding route-source validity changes the attribution of all 15 metric-bearing command-only records without changing their simulator outcome.

The timing results separate online and post-run costs. The 14.552 μ s value measures only camera/context and freshness checks inside prediction; the 234.855 μ s value measures in-memory diagnosis after telemetry parsing. A missing actor proxy or checkpoint remains a deployment precondition, a command proxy a semantic violation, and missing audit state an evidence violation. Typed outcomes keep those cases available for debugging while excluding them from policy attribution.

VI. RELATED WORK AND LIMITATIONS

The contracts instantiate interface reasoning for cyber-physical systems [3], [4]. Closed-loop platforms and benchmarks such as CARLA, nuPlan, Waymax, NAVSIM, and AlpaSim [5], [6], [7], [8], [2] emphasize simulation or planning evaluation. Scenario and falsification tools such as Scenic, VerifAI, DriveFuzz, and PlanFuzz [9], [10], [11], [12] generate or mutate conditions. WOD2Sim is complementary: it asks whether the policy/simulator boundary preserved the assumptions needed to interpret a rollout.

nuPlan and NAVSIM formalize planning tasks and measures; Waymax and CARLA provide controllable simulation interfaces; AlpaSim supplies the external-driver boundary used here. WOD2Sim instead applies contract assumptions and guarantees as runtime admission and artifact-lineage checks. Scenic and driving fuzzers vary scenarios to expose system behavior, whereas our mutations vary information or preconditions at the integration boundary. The approaches can be combined.

The study scope is contract conformance. The mutations are framework-authored and applied offline to one retained trace; the valid controls are correlated prefixes of that trace. The status-only gate is a concrete completion-and-metrics comparator, not a surrogate for CARLA, nuPlan, or another full framework. The repository is not an official WOMB interface, does not reconstruct full WOMB worlds, and has no public learned checkpoint, scene-matched actor proxy, or verified scenario-category coverage. Restricted scenes remain licensed inputs; public artifacts retain the trace, manifests, aggregates, hashes, and testable gate logic.

A. Threats to Validity

Internal validity. Mutation and detector code share the declared contracts, which can favor the expected fault vocabulary. To reduce circularity, construction and diagnosis are separate functions, expected labels are withheld from the detector, every case is scored afterward, and 15 unmutated controls test false alarms. McNemar’s test is paired by case; it measures the observed difference against the status gate on this designed set, not a population prevalence. Timing order is randomized and paired on one host, so hardware and background-load effects remain shared measurement conditions.

Construct validity. The online overhead ablation covers camera shape/context and freshness checks in dependency-light route following; the diagnosis timer begins after JSON parsing. All five contracts, including the plugin precondition within deployment, are mutated against retained telemetry and context rather than rerun as physical simulator faults. Contract-valid therefore means admitted to the stated evidence boundary, not driving safety or simulator correctness.

External and conclusion validity. The scenes were selected by local asset availability and remain category-unclassified. The diagnostic cases share one external trace, and the closed-loop route pairs have one execution per arm with no effective seed control. Accordingly, the reported comparison concerns the declared mutations and status gate; the measured latencies concern this host and dependency-light policy path. Learned-policy behavior, rare-event coverage, and route-loss effects require separate experiments.

VII. CONCLUSION

WOD2Sim formulates a closed-loop policy boundary as semantic, temporal, lifecycle, deployment, and evidence contracts. On 15 controlled faults and 15 valid controls it classifies 30/30 cases, localizes 15/15 faults, and differs from the status-only gate on all 15 fault cases. Post-trace diagnosis and online guard costs are measured rather than assumed. Together with the retained closed-loop route invalidations, the result shows that explicit gates make integration evidence auditable before a rollout is attributed to policy behavior.

ACKNOWLEDGMENT

OpenAI Codex (GPT-5) assisted with language editing, code audit, and test implementation. Repository scripts generated the measurements; the author reviewed the method, results, and manuscript and accepts responsibility for the content.

REFERENCES

- [1] S. Ettinger, S. Cheng, B. Caine, C. Liu, H. Zhao, S. Pradhan, Y. Chai, B. Sapp, C. R. Qi, Y. Zhou, Z. Yang, A. Chouard, P. Sun, J. Ngiam, V. Vasudevan, A. McCauley, J. Shlens, and D. Anguelov, “Large scale interactive motion forecasting for autonomous driving: The Waymo open motion dataset,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 9710–9719, Oct. 2021.
- [2] NVIDIA, Y. Cao, R. de Lutio, S. Fidler, G. Garcia Cobo, Z. Gojcic, M. Igl, B. Ivanovic, P. Karkus, J. Martinez Esturo, M. Pavone, A. Smith, E. Tanimura, M. Tyszkiewicz, M. Watson, Q. Wu, and L. Zhang, “AlpaSim: A modular, lightweight, and data-driven research simulator for autonomous driving,” Software, Oct. 2025.
- [3] A. Sangiovanni-Vincentelli, W. Damm, and R. Passerone, “Taming Dr. Frankenstein: Contract-based design for cyber-physical systems,” *European Journal of Control*, vol. 18, no. 3, pp. 217–238, 2012.
- [4] L. de Alfaro and T. A. Henzinger, “Interface automata,” in *Proceedings of the 8th European Software Engineering Conference*, pp. 109–120, ACM, 2001.
- [5] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, “CARLA: An open urban driving simulator,” in *Proceedings of the 1st Annual Conference on Robot Learning*, vol. 78 of *Proceedings of Machine Learning Research*, pp. 1–16, PMLR, 2017.
- [6] H. Caesar, J. Kabzan, K. S. Tan, W. K. Fong, E. Wolff, A. Lang, L. Fletcher, O. Beijbom, and S. Omari, “nuPlan: A closed-loop ML-based planning benchmark for autonomous vehicles,” in *CVPR 2021 ADP3 Workshop*, 2021.
- [7] C. Gulino, J. Fu, W. Luo, G. Tucker, E. Bronstein, Y. Lu, J. Harb, X. Pan, Y. Wang, X. Chen, J. D. Co-Reyes, R. Agarwal, R. Roelofs, Y. Lu, N. Montali, P. Mouglin, Z. Yang, B. White, A. Faust, R. McAllister, D. Anguelov, and B. Sapp, “Waymax: An accelerated, data-driven simulator for large-scale autonomous driving research,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023. Datasets and Benchmarks Track.
- [8] D. Dauner, M. Hallgarten, T. Li, X. Weng, Z. Huang, Z. Yang, H. Li, I. Gilitschenski, B. Ivanovic, M. Pavone, A. Geiger, and K. Chitta, “NAVSIM: Data-driven non-reactive autonomous vehicle simulation and benchmarking,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024. Datasets and Benchmarks Track.
- [9] D. J. Fremont, T. Dreossi, S. Ghosh, X. Yue, A. L. Sangiovanni-Vincentelli, and S. A. Seshia, “Scenic: A language for scenario specification and scene generation,” in *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pp. 63–78, ACM, 2019.
- [10] T. Dreossi, D. J. Fremont, S. Ghosh, E. Kim, H. Ravanbakhsh, M. Vazquez-Chanlatte, and S. A. Seshia, “VerifAI: A toolkit for the formal design and analysis of artificial intelligence-based systems,” in *Computer Aided Verification*, vol. 11561 of *Lecture Notes in Computer Science*, pp. 432–442, Springer, 2019.
- [11] S. Kim, M. Liu, J. Rhee, Y. Jeon, Y. Kwon, and C. H. Kim, “DriveFuzz: Discovering autonomous driving bugs through driving quality-guided fuzzing,” in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pp. 1753–1767, ACM, Nov. 2022.
- [12] Z. Wan, J. Shen, J. Chuang, X. Xia, J. Garcia, J. Ma, and Q. A. Chen, “Too afraid to drive: Systematic discovery of semantic DoS vulnerability in autonomous driving planning under physical-world attacks,” in *Proceedings of the Network and Distributed System Security Symposium*, Internet Society, 2022.